

Claude Code Routines: desligando o Mac mini sem colocar a main em risco

Resumo pessoal sobre por que o Claude Code Routines não é "só um cron com outro nome" e quais travas nativas impedem a IA de quebrar a produção rodando sozinha de madrugada — a partir da prática "O Especialista em Automação e o Fundador Refém do Mac Mini Ligado".

Fernando Melo · 14 de setembro de 2026

01 · O PROBLEMA

Tarefa repetitiva, Mac mini ligado a noite toda

O ponto de partida foi bem prático: triagem de bug toda manhã, scan de dependências toda semana, e um Mac mini ligado a noite inteira rodando crons que "vivem quebrando". A pergunta central era dupla — isso não é só um cron com outro nome? E o principal medo: deixar uma IA operando sozinha, de madrugada, sem ninguém olhando, mexendo direto no repositório de produção.

02 · POR QUE NÃO É "SÓ UM CRON"

Três diferenças de concepção

Raciocínio adaptativo

Em vez de rodar um comando estático, a instrução é em linguagem natural. O Claude analisa o contexto atual e toma decisões em tempo real, em vez de simplesmente disparar um script cego.

Infraestrutura em nuvem

As rotinas rodam na infraestrutura gerenciada da Anthropic, clonando o repositório em um ambiente limpo a cada execução — o Mac mini pode ficar desligado.

Múltiplos gatilhos

Além do agendamento por horário, uma rotina pode ser disparada por eventos do GitHub (ex.: abertura de PR) ou por chamadas de API (webhooks).

Resultado prático

Um cron tradicional quebra se o ambiente muda 1 milímetro; a Routine lê o erro e tenta caminhos alternativos.

03 · O QUE IMPEDE A IA DE DESTRUIR A MAIN

Travas de segurança nativas

- 01 Regra do prefixo claude/: por padrão, é proibido fazer push direto para branches principais ou protegidas — toda alteração vai para uma branch nova com esse prefixo.
- 02 Bloqueio de ramificações protegidas: se o prompt ordenar um push direto na main, o ecossistema intercepta, valida as regras de proteção do GitHub e rejeita o push automaticamente.
- 03 Trabalho via Pull Request: o fluxo ideal para IA autônoma nunca dá a ela poder de commitar direto em produção — ela altera a branch isolada e abre um PR para revisão humana.
- 04 Sandboxing e escopo limitado: cada execução roda em container temporário e limpo, com acesso restrito manualmente a conectores (MCP) específicos, como só Slack ou só Jira.

Diferença prática lado a lado

CARACTERÍSTICA	MAC MINI COM CRON	CLAUDE CODE ROUTINES
Onde roda	Máquina física (precisa ficar ligada)	Nuvem da Anthropic
Resiliência a falhas	Baixa (quebra se a saída do log mudar)	Alta (lê o erro e tenta caminhos alternativos)
Escrita de código	Impossível (só roda scripts fixos)	Capaz de gerar código, abrir PRs e corrigir vulnerabilidades
Risco para a main	Alto (basta um git push -f)	Bloqueado por padrão (exige prefixo claude/)

Triagem de bugs às 7h da manhã

Trecho ilustrativo simplificado

Gatilho: Cron (Seg a Sex, 07:00 BRT)
Instrução:
1. Buscar Issues de "Bug" abertas nas últimas 24h.
2. Analisar o código para localizar arquivo e linha do erro.
3. Se houver correção óbvia: abrir PR em modo Draft.
4. Enviar resumo para o canal #dev-bugs no Slack.

Enquanto o desenvolvedor dorme, o log é totalmente auditável: a rotina conecta ao GitHub, identifica a issue, varre o código, cria a branch claude/fix-login-nullpointer, abre o PR de rascunho e envia o relatório ao Slack. Ao acordar, existe uma notificação resumida e um PR pronto para revisão — o controle final de aprovar ou fechar continua sendo humano.

A regra dos 85% de certeza

Para evitar que a IA crie PRs inúteis (ruído em vez de ajuda), a estratégia é configurar travas explícitas de confiança no próprio prompt da rotina.

Confiança ≥ 85%

Cria a branch claude/fix-..., abre o PR como Draft e explica a correção na descrição.

Confiança < 85%

Não cria branch, não abre PR. Apenas deixa um comentário técnico na Issue original explicando a suspeita e o que falta para confirmar.

No pior cenário — um PR completamente errado — o custo extra é de segundos: fechar o Pull Request e seguir. Quando o bug é ambíguo, o comentário na Issue já economiza tempo de investigação sem tocar em uma linha de código.

Sandbox e o "teste do labirinto"

- 01 Criar um repositório privado "espelho" (ex.: meu-projeto-sandbox), com a estrutura básica do código e sem dados sensíveis.
- 02 Configurar o Claude Code com acesso apenas a esse repositório sandbox.
- 03 Disparar a rotina manualmente pelo botão "Run Now" e acompanhar o log ao vivo, em vez de esperar a execução agendada.

Cenário A · Bug óbvio

Erro bobo e claro no código. Esperado: certeza >85%, branch criada e PR aberto.

Cenário B · Bug ambíguo

Issue vaga, sem logs. Esperado: certeza <85%, sem branch/PR — apenas comentário apontando caminhos.

Cenário C · Alarme falso — issue sem sentido ou de infraestrutura externa (ex.: "a API do banco caiu"). Esperado: a IA ignora ou apenas comenta que o problema não está no código do repositório.

Prompt de QA autônomo para o sandbox

Trecho ilustrativo simplificado

Papel: Engenheiro de QA e Triagem autônomo (repositório sandbox)

1. Ler issues das últimas 24h com a label "bug".
2. Localizar arquivo/função/linha da causa raiz.
3. Avaliar confiança de 0 a 100%:
 - >= 85%: branch claude/fix-issue-N + PR Draft.
 - < 85%: apenas comentário técnico na Issue.
 - fora de escopo: comentar e encerrar.
4. Nunca commitar/push direto em main, master ou branch protegida.
5. Ao final, enviar resumo consolidado ao Slack/Teams.

Schedule, API e eventos do GitHub

Agendado (Schedule)

Funciona como um cron inteligente — de hora em hora, diário, em dias úteis ou semanal. Ideal para tarefas de rotina sem urgência.

Por API (Webhook)

Cada rotina recebe um endpoint e token próprios, plugável em alertas de monitoramento (Datadog, Sentry) ou hooks de deploy.

Combinação prática — "toda noite às 2h (agendado), ou sempre que um alerta crítico disparar (via API), o Claude pega o bug mais urgente e abre um PR em rascunho na branch isolada." Também é possível reagir a eventos do GitHub, como a abertura de um PR, para revisões automáticas.

Como levar isso para o time com os pés no chão

O Claude Code Routines ainda está em fase de Research Preview: o comportamento do modelo é refinado constantemente, e workflows pesados ou muito frequentes consomem mais cota de uso. A recomendação é começar pequeno, em tarefas de baixo risco e alto valor.

CENÁRIO	RISCO	VALOR ENTREGUE
Triagem diária de bugs (lendo issues)	Muito baixo	Alto
Scan de dependências semanal (PR draft)	Baixo	Alto
Refatoração automática direto na main	Altíssimo	Instável

Próximo passo — com o sandbox criado e o prompt de triagem pronto, rodar a rotina agendada uma vez por dia (ou via "Run Now") já dá visibilidade de custo e prova o valor antes de qualquer automação tocar o repositório principal.